

4 Netzwerkstrategien und ihre Bewertung

Im dynamischen Bereich des Deep Learnings spielen das Design, die Optimierung und die Bewertung neuronaler Netze eine entscheidende Rolle beim Erreichen von Hochleistungsmodellen. Dieses Kapitel bietet einen umfassenden Überblick über wesentliche Strategien zum Aufbau und zur Bewertung tiefer Netze, wobei der Schwerpunkt auf Architekturauswahl, Aktivierungsfunktionen, Optimierungsmethoden, Regularisierungstechniken und Modellbewertungsmetriken liegt.

Wichtige Überlegungen beim Architekturdesign, wie Schichtkonfiguration, Merkmalskalierbarkeit und rechnerische Effizienz, leiten die Wahl der Netzwerkstrukturen, während Fortschritte wie die Neuronale Architektursuche (NAS) den Prozess automatisieren. Die Auswahl der Aktivierungsfunktionen, wie ReLU, Sigmoid und Tanh, ist entscheidend für die Bestimmung der Lernfähigkeit eines Netzwerks, wobei Alternativen wie Leaky ReLU und Mish eine bessere Gradientenstabilität bieten. Optimierungsmethoden, insbesondere Stochastic Gradient Descent (SGD) und Adam, verfeinern den Lernprozess, wobei Hybridansätze wie SWATS die Konvergenzgeschwindigkeit und Generalisierung ausgleichen. Regularisierungstechniken wie Dropout und L2-Gewichtszerrfall sind entscheidend, um Overfitting zu kontrollieren und die Modellrobustheit zu verbessern. [73] Zur Bewertung der Modellleistung liefern Metriken wie Mean Squared Error, R^2 -Score, Genauigkeit, Präzision und Recall Einblicke in die Vorhersagegenauigkeit, während Kreuzvalidierungstechniken und Early Stopping dazu beitragen, Overfitting zu mindern und die Generalisierung auf ungesehene Daten zu verbessern. Dieses Kapitel befasst sich mit diesen Strategien und bietet eine ganzheitliche Sicht darauf, wie Netzwerke entworfen, trainiert und bewertet werden, um eine optimale Leistung zu erzielen. [18]

4.1 Architekturauswahl und Designkriterien

Die Effektivität von Deep-Learning-Modellen wird maßgeblich durch ihr architektonisches Design bestimmt, das traditionell durch Expertenwissen und manuelle Abstimmung geleitet wird. Das Aufkommen der **Neuronalen Architektursuche (NAS)** hat diesen Prozess jedoch revolutioniert, indem es die Auswahl und Optimierung von Netzwerkstrukturen automatisiert. Bei der Bewertung oder dem Entwurf von Architekturen müssen mehrere Schlüsselkriterien berücksichtigt werden, um Effizienz, Skalierbarkeit und Generalisierung zu gewährleisten. [81]

Im Folgenden werden wir uns die neuronalen Netze (NN) genauer ansehen, einschließlich der Eingabe-, versteckten und Ausgabeschichten. Die Neuronen in aufeinanderfolgenden Schichten sind vollständig verbunden. Die Eingabedaten werden durch $\{x_1, x_2, \dots, x_n\}$ und die Ausgabedaten durch $\{y_1, y_2, \dots, y_m\}$ dargestellt. Die Ausgabedaten jeder Schicht werden als $\{h_1(l), h_2(l), \dots, h_{n_l}(l)\}$ dargestellt, wobei das Gewicht und der Bias für jedes Neuron als $w_{ii}(l)$ bzw. $b_{ii}(l)$, bezeichnet werden. [19]

Die Sigmoid- und Swish-Aktivierungsfunktionen werden verwendet, um die Lernfähigkeit des Netzwerks zu verbessern. Zusätzlich wird Dropout angewendet, um Overfitting zu verhindern, und Batch-Normalisierung wird verwendet, um ein stabiles Training zu gewährleisten. Das Netzwerk wird mit dem Stochastic Gradient Descent (SGD)-Algorithmus trainiert. Schließlich werden die NN-Parameter, einschließlich der Anzahl der Schichten, der Lernrate, der Dropout-Rate und anderer Einstellungen, in Tabelle 4.1 dargestellt.

Number	Parameters	Value
1	Full connection layer	3
2	Learning rate	0.001
3	Dropout	0.5
4	Epoch	10,000
5	Batch size	1024
6	Adam	Stochastic Gradient Descent

Diese Tabelle zeigt die wichtigsten Einstellungen, die für das Training des KI-Modells verwendet werden, wie ein Rezept für das Lernen. Die KI verwendet drei Schichten zur Verarbeitung von Informationen, lernt langsam (0,001), um Fehler zu vermeiden, und „vergisst“ während des Trainings zufällig die Hälfte ihres Wissens (0,5 Dropout), um flexibel zu bleiben. Es übt 10.000 Runden lang an jeweils 1.024 Beispielen und nutzt intelligente, selbstanpassende Techniken, um Aufgaben wie die Bilderkennung effizient zu meistern. Basierend auf der Forschung von "Learning Activation Functions to Improve Deep Neural Networks" (2019). [72]

1. Suchraumgestaltung Der Suchraum definiert den Bereich möglicher Architekturen, wobei Flexibilität und Handhabbarkeit ausbalanciert werden. Gängige Designentscheidungen umfassen:

- **Kettenstrukturierte Netzwerke** oder auch sequenzielle Schichten werden zur Vereinfachung und einfacheren Implementierung verwendet.
- **Mehrzweig-Topologien** zur Verbesserung des Gradientenflusses und der Merkmalswiederverwendung.
- **Zellenbasierte Architekturen**, bei denen modulare Bausteine gestapelt werden, um die Skalierbarkeit und Übertragbarkeit zu verbessern. [70]

Ein gut strukturierter Suchraum integriert vorhandenes Wissen, während er genügend Flexibilität beibehält, um Innovationen zu fördern.

2. Leistungsgesteuerte Auswahlstrategien Die Auswahl einer optimalen Architektur erfordert eine effiziente Navigation im Suchraum unter Verwendung verschiedener Optimierungsstrategien:

- **Reinforcement Learning (RL):** Verwendet eine politikgesteuerte Architekturgenerierung, um die Validierungsgenauigkeit zu maximieren.
- **Evolutionäre Algorithmen:** Verwenden Mutations- und Selektionstechniken, um Architekturen zu verfeinern, wodurch oft kompakte, hochleistungsfähige Modelle entstehen.

- **Gradientenbasierte Methoden:** Nutzen kontinuierliche Relaxationen, um eine differenzierbare Architektursuche zu ermöglichen. [70]

Eine effektive Auswahlstrategie muss die Exploration und Exploitation (Verfeinerung bekannter hochleistungsfähiger Architekturen) ausbalancieren.

3. Effizienz und Ressourcenbeschränkungen Die praktische Architekturauswahl beinhaltet die Bewertung von Kompromissen zwischen:

- **Genauigkeit vs. Rechenkosten:** Niedrige Wiedergabetreue-Schätzungen oder Gewichtsteilungstechniken beschleunigen die Bewertung, können aber Verzerrungen einführen.
- **Skalierbarkeit:** Hierarchische Suchräume verbessern die Anpassungsfähigkeit an verschiedene Aufgaben.
- **Hardware-Kompatibilität:** Architekturen müssen den Bereitstellungsbeschränkungen, wie Latenz- und Speicherbegrenzungen, entsprechen. [71]

4. Generalisierung und Robustheit Über die aufgabenspezifische Leistung hinaus sollten ideale Architekturen:

- Effektiv auf neue Datensätze oder Domänen übertragen werden.
- Overfitting durch Modularität und Regularisierung widerstehen.
- Sich dynamisch an Multi-Task- oder Multi-Objective-Einstellungen anpassen, wobei Genauigkeit und Effizienz ausbalanciert werden. [71]

5. Der Aufstieg der Neuronalen Architektursuche (NAS) Das traditionelle Design neuronaler Netze war lange Zeit von manuellem Fachwissen abhängig, was oft zu suboptimalen oder rechenintensiven Modellen wie ResNet und Inception führte. Die **Neurale Architektursuche (NAS)** automatisiert diesen Prozess, indem sie Architekturen algorithmisch untersucht, um die Leistung innerhalb vordefinierter Suchräume zu optimieren. Schlüssel-NAS-Methoden umfassen:

- **Reinforcement Learning (RL):** Strategien generieren Architekturen, die basierend auf der Validierungsgenauigkeit belohnt werden.
- **Evolutionäre Algorithmen:** Architekturen entwickeln sich durch Mutation und Selektion und verfeinern Modelle iterativ.
- **Zufallssuche:** Trotz ihrer Einfachheit können gut strukturierte Suchräume überraschend effektive Ergebnisse liefern. [19] Durch die Eliminierung der Notwendigkeit manueller Abstimmung beschleunigt NAS die Modellfindung und verbessert die Effizienz in verschiedenen Deep-Learning-Anwendungen.

6. Hybride Ansätze: Kombination von Automatisierung mit menschlichem Fachwissen [82] Um Effizienz und Innovation auszubalancieren, integrieren Hybridmethoden NAS mit manuellen Designprinzipien:

- **Hierarchische Suchräume:** Inspiriert von menschlich entworfenen modularen Motiven, die skalierbare und übertragbare Architekturen ermöglichen.
- **Beschränkte Suche:** Begrenzt die Exploration auf plausible Topologien (z. B. vordefinierte Makrostrukturen), um die Rechenkosten zu senken.
- **Warm-Start NAS:** Initialisiert Suchen mit hochleistungsfähigen handgefertigten Architekturen (z. B. ResNet-basierten Zellen), um die Konvergenz zu beschleunigen. [18] Diese Ansätze stellen sicher, dass NAS von der menschlichen Intuition profitiert, während die Vorteile der Automatisierung erhalten bleiben.

7. Realweltanwendungen in der Industrie NAS wurde erfolgreich in verschiedenen Branchen eingesetzt, um Herausforderungen im Zusammenhang mit Skalierbarkeit und Ressourceneffizienz zu bewältigen:

- **Computer Vision:**
 - **Bildklassifizierung:** NAS-entworfene Modelle übertreffen manuell erstellte Architekturen bei Benchmarks wie CIFAR-10 und ImageNet.
 - **Objekterkennung & Segmentierung:** AutoML-Systeme optimieren Architekturen für Edge-Geräte, wobei Latenz und Genauigkeit ausbalanciert werden.
- **Natural Language Processing:** Die automatisierte Architekturentdeckung verbessert Transformer- und RNN-Modelle für textbasierte Anwendungen.
- **Hardware-Aware NAS:** Benutzerdefinierte Architekturen werden für TPUs, GPUs und eingebettete Systeme angepasst, um die Leistung für spezifische Hardware-Beschränkungen zu optimieren. [81]

4.2 Aktivierungsfunktionen: ReLU, Sigmoid, Tanh

In künstlichen neuronalen Netzen spielen Aktivierungsfunktionen eine entscheidende Rolle bei der Gestaltung des Lernprozesses und der Beeinflussung der Gesamtleistung. Während viele Aktivierungsfunktionen als universelle Approximatoren dienen, variiert ihre Effizienz, was umfangreiche Forschungsarbeiten zur Verbesserung der Fähigkeiten neuronaler Netze anregt. Trotz ihrer Bedeutung stoßen konventionelle Aktivierungsfunktionen oft auf Herausforderungen wie Sättigung, das "sterbende Neuron"-Problem und das Problem des explodierenden oder verschwindenden Gradienten.

Um diese Einschränkungen zu beheben, stellt diese Arbeit zwei neuartige Aktivierungsfunktionen vor – absolute lineare Einheiten und inverse polynomische lineare Einheiten – die jeweils einen einstellbaren Parameter enthalten, um die Steigung des Gradienten für eine verbesserte Optimierung zu optimieren. Diese Studie bietet auch eine detaillierte Taxonomie von Aktivierungsfunktionen, die deren Eigenschaften und Verhalten systematisch kategorisiert. Umfassende Experimente werden an verschiedenen tiefen neuronalen Architekturen durchgeführt, wobei Netzwerktyp und -tiefe berücksichtigt werden, um die Wirksamkeit dieser neuartigen Funktionen zu bewerten. Ihre Leistung wird bei Bild- und Textklassifizierungsaufgaben unter Verwendung mehrerer öffentlicher Benchmark-Datensätze streng getestet, um einen gründlichen

Vergleich mit weit verbreiteten Aktivierungsfunktionen zu gewährleisten. Die Ergebnisse zeigen, dass die vorgeschlagenen Aktivierungsfunktionen viele konventionelle in mehreren Benchmarks übertreffen. Eine statistische Analyse über Klassifizierungskategorien hinweg bestätigt zusätzlich ihre Robustheit und überragende Genauigkeit und etabliert sie als vielversprechende Alternativen in Deep-Learning-Anwendungen. [18]

1. Grundlegende Aktivierungsfunktionen: Sigmoid, Tanh und ReLU

Aktivierungsfunktionen führen Nichtlinearität in neuronale Netze ein und ermöglichen es ihnen, komplexe Muster zu lernen. Zu den am weitesten verbreiteten gehören die Rectified Linear Unit (ReLU), die logistische Sigmoid-Funktion und die Hyperbolic Tangent (Tanh)-Funktion – jede bietet einzigartige Vorteile und Kompromisse. [77]

Traditionelle Nichtlinearitäten: Sigmoid und Tanh Historisch waren Sigmoid- und Tanh-Funktionen grundlegend für die Einführung von Nichtlinearität.

- **Sigmoid-Funktion:** Die Sigmoid-Aktivierungsfunktion bildet Eingangswerte auf den Bereich $[0,1]$ ab, was sie für probabilistische Interpretationen in Ausgabeschichten nützlich macht. Sie ist definiert als:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Obwohl historisch beliebt, leidet Sigmoid unter verschwindenden Gradienten bei extremen Eingangswerten, was das Training tiefer Netze aufgrund von Sättigung erheblich verlangsamt.

- **Hyperbolic Tangent (Tanh)-Funktion:** Die Tanh-Funktion transformiert Eingaben in den Bereich $[-1,1]$, wodurch sie nullzentriert wird, was die Konvergenz im Vergleich zu Sigmoid unterstützen kann. Sie ist gegeben durch:

$$\text{Tanh}(x) = \frac{e^x + e^{-x}}{e^x - e^{-x}}$$

Tanh verbessert die Konvergenz gegenüber Sigmoid, leidet aber immer noch unter Gradientensättigung bei großen Eingabewerten, ähnlich der Sigmoid-Funktion.

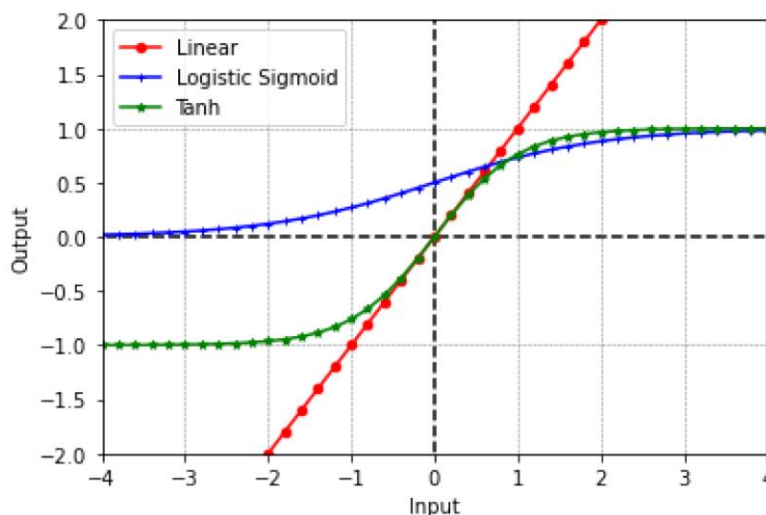


Abb. 4.1 für eine Illustration von linearen, logistischen Sigmoid- und Tanh-Aktivierungsfunktionen, angepasst aus "Activation Functions in Deep Learning" (2022) [18]).

2. Der moderne Standard: Rectified Linear Unit (ReLU) Die Rectified Linear Unit (ReLU) ist aufgrund ihrer Recheneffizienz und ihrer Fähigkeit, Sättigung für positive Eingaben zu vermeiden, weit verbreitet im Deep Learning. Sie ist definiert als:

$$\text{ReLU}(x) = \max(0, x)$$

Vorteile von ReLU:

- **Effiziente Berechnung:** ReLU beinhaltet keine Exponentialfunktion, wodurch die Berechnung wesentlich schneller erfolgt als bei Sigmoid oder Tanh.
- **Sparsität:** Durch die Ausgabe von Null für negative Eingaben führt ReLU Sparsität im Netzwerk ein, was potenziell die Aktivierungsredundanz reduziert und die Recheneffizienz verbessert. [74]

Einschränkungen von ReLU:

- **Sterbende ReLU (Dying ReLU):** Neuronen können dauerhaft inaktiv werden, wenn ihre Eingabe konstant in den negativen Bereich fällt, was zu einem Gradienten von Null führt. Dies bedeutet, dass das Neuron das Lernen vollständig einstellt.
- **Unbegrenzte Ausgabe:** Für positive Eingaben ist die Ausgabe von ReLU unbegrenzt, was manchmal zu Instabilität in sehr tiefen Netzen führen kann, indem es große Aktivierungswerte verursacht. [74]

3. Hybride und adaptive Varianten Um die Mängel von ReLU zu mildern und die Einschränkungen traditioneller Aktivierungsfunktionen zu beheben, wurden mehrere modifizierte und adaptive Varianten eingeführt:

- **Leaky ReLU (LReLU):** Diese Variante behebt das Problem des "sterbenden ReLU", indem sie einen kleinen, nicht-null Gradienten für negative Eingaben zulässt. Sie ist definiert als:

$$\text{Leaky ReLU}(x) = \begin{cases} x & \text{falls } x > 0 \\ \alpha x & \text{falls } x \leq 0 \end{cases} \text{ wobei } \alpha \text{ eine kleine positive Konstante ist. [18]}$$

- **Parametric ReLU (PReLU):** Eine Erweiterung von Leaky ReLU, PReLU verwendet einen lernbaren Parameter α für negative Eingaben, wodurch das Netzwerk die Steigung während des Trainings anpassen kann.
- **Scaled Tanh & SiLU (Sigmoid Linear Unit):** Diese und andere Varianten zielen darauf ab, die Vorteile von Sigmoid/Tanh (z. B. begrenzte Ausgaben) zu kombinieren, während ein effizienter Gradientenfluss erhalten bleibt und Sättigungsprobleme behoben werden. [72]

4. Praktische Überlegungen

- **Prävalenz von ReLU:** ReLU wird aufgrund seiner Einfachheit und Effektivität im Allgemeinen für die meisten Deep-Learning-Anwendungen bevorzugt, da es ein gutes Gleichgewicht zwischen Rechenkosten und Leistung bietet.
- **Kontextspezifische Nutzung:** Sigmoid und Tanh bleiben in bestimmten Kontexten nützlich, wie z.B. bei der binären Klassifizierung (Sigmoid in der Ausgabeschicht) und rekurrenten neuronalen Netzen (Tanh), erfordern aber eine sorgfältige Initialisierung, um verschwindende Gradienten zu vermeiden.
- **Aufkommende Funktionen:** Die Entwicklung neuartiger Aktivierungsfunktionen, einschließlich der in dieser Arbeit vorgeschlagenen absoluten linearen Einheiten und inversen polynomischen linearen Einheiten, integriert weiterhin Eigenschaften bestehender Funktionen, um deren Einschränkungen zu überwinden. Während diese neuen Funktionen vielversprechende Ergebnisse zeigen, beeinflussen ihre Rechenkosten und die einfache Integration in bestehende Frameworks oft ihre weitreichende Akzeptanz. [18]

4.3 Optimierungsmethoden: SGD, Adam

Trotz ihrer starken Leistung während des frühen Trainings können adaptive Optimierungsmethoden wie Adam, Adagrad und RMSprop in den späteren Trainingsphasen oft nicht mit der Generalisierungsfähigkeit von Stochastic Gradient Descent (SGD) mithalten. Während diese Methoden anfänglich die Konvergenz beschleunigen, werden sie in Bezug auf die endgültige Modellleistung tendenziell von SGD übertroffen. Um dies zu beheben, wurde ein hybrider Ansatz untersucht – beginnend mit einer adaptiven Methode und dem Wechsel zu SGD im richtigen Moment. Eine solche Strategie, SWATS (Switching from Adam to SGD), führt einen einfachen Mechanismus ein, der von Adam zu SGD übergeht, basierend auf einer Bedingung, die aus der Ausrichtung der Optimierungsschritte mit dem Gradientenunterraum abgeleitet wird. Dieser Wechsel verursacht minimalen zusätzlichen Rechenaufwand und erfordert keine zusätzlichen Hyperparameter. [73]

1. Stochastic Gradient Descent (SGD) SGD ist ein grundlegender Optimierungsalgorithmus, der zum Training tiefer neuronaler Netze verwendet wird, indem die Verlustfunktion $f(w)$ iterativ minimiert wird. Die Regel lautet:

$$w_k = w_{k-1} - \eta \nabla f(w_{k-1})$$

wobei η die Lernrate ist und ∇f den stochastischen Gradienten darstellt. SGD wird aufgrund seiner Einfachheit und theoretischen Garantien, wie der Vermeidung von Sattelpunkten und seiner Bayes'schen Interpretation, weit verbreitet eingesetzt. Seine gleichmäßige Skalierung des Gradienten über alle Parameter kann jedoch bei schlecht konditionierten Problemen zu einer langsamen Konvergenz führen. [76]

Momentumbasiertes SGD (SGDM) Momentum verbessert SGD durch die Einbeziehung eines Geschwindigkeitsterms v_t , der hilft, die Gradientenaktualisierungen zu glätten:

$$w_k = w_k - \beta - \eta k - 1 v_k w_k = w_{\{k-1\}} - \eta_{\{k-1\}} v_k$$

wobei $\beta \in [0,1)$ der Momentumkoeffizient ist. Diese Technik beschleunigt die Konvergenz durch Reduzierung von Oszillationen in der Optimierungstrajektorie.

2. Adaptive Methoden: Adam Adam (Adaptive Moment Estimation) verbessert SGD, indem es die Lernraten pro Parameter unter Verwendung von Schätzungen der Gradientenmomente anpasst. Die Aktualisierungsregeln sind:

$$w_k = w_k - \beta_1 - \eta k - 1 m_k - 1 v_k - 1 + \epsilon w_k = w_{\{k-1\}} - \eta_{\{k-1\}} \frac{m_{\{k-1\}}}{\sqrt{\{v_{\{k-1\}}\} + \epsilon}}$$

wobei β_1 und β_2 die exponentiellen Abklingraten steuern und ϵ eine kleine Konstante ist, um eine Division durch Null zu verhindern. Adam ist besonders effektiv für spärliche Gradienten und schlecht skalierte Probleme, kann jedoch in einigen Fällen aufgrund der nicht-uniformen Gradientenskalierung schlechter generalisieren als SGD. [74]

3. Hybride Ansätze Um Adams schnellen initialen Fortschritt mit SGDs überlegener Generalisierung zu kombinieren, wechselt SWATS (Switching from Adam to SGD) automatisch von Adam zu SGD:

- **Schätzung der Lernrate:** Projiziert Adams Schritt auf den Gradientenunterraum, um eine SGD-kompatible Rate abzuleiten.
- **Wechselkriterium:** Löst aus, wenn sich die bias-korrigierte Adam-Rate stabilisiert.

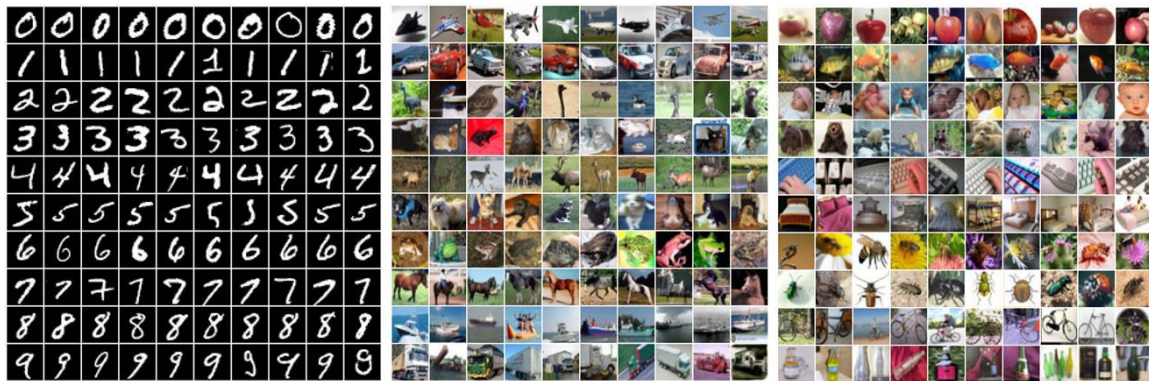
Empirische Studien zeigen, dass SWATS Adams Effizienz in der Frühphase beibehält und gleichzeitig SGDs Endleistung erreicht. [74]

4. Praktische Einblicke

- **SGD/SGDM** zeichnet sich bei generalisierungskritischen Aufgaben aus, erfordert jedoch eine sorgfältige Abstimmung der Lernrate.
- **Adam** ist robust gegenüber Hyperparametern und ideal für frühe Trainingsphasen.
- **Adaptive Methoden** bieten Kompromisse zwischen Konvergenzgeschwindigkeit und Generalisierung. [73]

4.4 Regularisierung: Dropout und L2-Regularisierung

Regularisierungstechniken sind entscheidend, um Overfitting in tiefen neuronalen Netzen zu mildern, insbesondere wenn die Trainingsdaten begrenzt sind. Sie verbessern die Modellgeneralisation, insbesondere beim Training auf komplexen und



hochdimensionalen Daten. Dieser Abschnitt untersucht zwei weit verbreitete Methoden: Dropout und L2-Regularisierung, die beide erheblich zur Verbesserung der Generalisierungsfähigkeit neuronaler Netze beitragen und es ihnen ermöglichen, auf ungesehenen Daten gut zu funktionieren. [78]

Abb. 4.2: Beispielbilder aus den Datensätzen MNIST, CIFAR-10 und CIFAR-100.

Jede Zeile zeigt Bilder einer bestimmten Kategorie. (Adaptiert aus „Towards Dropout Training for Convolutional Neural Networks“ (2015) [77].

1. Dropout: Stochastische Regularisierung Dropout ist eine leistungsstarke Regularisierungstechnik, die Overfitting mindert, indem sie während des Trainings einen Bruchteil von Neuronen zufällig deaktiviert. Dies verhindert die Ko-Adaptation zwischen Merkmalen und trainiert effektiv ein Ensemble von Teilnetzwerken.

- **Mechanismus:** Während jeder Trainingsiteration setzt Dropout zufällig einen Bruchteil der Neuronenaktivierungen auf Null.
- **Effekt:** Diese stochastische Deaktivierung zwingt das Modell, robustere Merkmale zu lernen und verhindert, dass es sich zu stark auf ein einzelnes Neuron verlässt.
- **Anwendung:** Dropout ist besonders nützlich in großen neuronalen Netzen, wo das Risiko von Overfitting aufgrund einer übermäßigen Anzahl von Parametern hoch ist.
- **Trainingsphase:** Für jeden Mini-Batch werden Neuronen stochastisch "gedroppt", und die Aktivierungen der verbleibenden Neuronen werden mit $1/p$ (wobei p die Beibehaltungswahrscheinlichkeit ist) skaliert, um die erwarteten Größen der Netzwerkausgabe beizubehalten.
- **Inferenzphase:** Alle Neuronen bleiben aktiv, aber ihre Gewichte werden mit p (oder Aktivierungen mit $1/p$) skaliert, um die Modellmittelung zu approximieren, die charakteristisch für ein Ensemble ist. [78]

2. Varianten von Dropout:

- **Max-Pooling Dropout:** Diese Variante wendet Dropout auf Pooling-Regionen an, was als Stichprobe aus einer multinomialen Verteilung interpretiert werden kann.
- **Probabilistic Weighted Pooling:** Diese Methode ersetzt deterministisches Max-Pooling zur Testzeit durch eine gewichtete Summe von Aktivierungen, was oft zu einer verbesserten Generalisierung führt.

Empirische Studien zeigen, dass Dropout Overfitting in vollständig verbundenen Schichten erheblich reduziert. Seine Wirksamkeit in Faltungsschichten kann jedoch aufgrund der inhärenten Parameterteilung und räumlichen Lokalität begrenzt sein. [19]

3. L2-Regularisierung: Gewichtszerfall L2-Regularisierung, auch bekannt als Gewichtszerfall, ist eine weit verbreitete Regularisierungstechnik, die große Gewichte in einem neuronalen Netzwerk bestraft.

- **Mechanismus:** L2-Regularisierung fügt der Verlustfunktion einen Term hinzu, der auf der quadrierten Größe der Gewichte basiert. Die modifizierte Verlustfunktion ist gegeben durch:

$Loss_{new} = Loss_{original} + \lambda \sum w_i^2$ wobei λ (Lambda) ein Hyperparameter ist, der die Stärke der Strafe steuert, und w_i die einzelnen Gewichte im Netzwerk darstellt.

- **Effekt:** Diese Strafe ermutigt das Modell, glattere, kleinere Gewichte zu lernen, wodurch die Modellkomplexität reduziert und die Generalisierung verbessert wird. Durch das Verhindern großer Gewichte verhindert die L2-Regularisierung, dass das Modell Rauschen in den Trainingsdaten anpasst, was zu stabileren Modellen führt. [19]

4. Vergleichende Analyse und Hybridansatz Dropout und L2-Regularisierung sind komplementäre Techniken zur Bekämpfung von Overfitting, und ihre kombinierte Verwendung führt oft zu einer überlegenen Leistung.

- **Stärken von Dropout:** Dropout zeichnet sich in Hochkapazitätsnetzwerken aus und erzielt mit Benchmarks wie MNIST und CIFAR-10 modernste Ergebnisse, hauptsächlich durch die Verhinderung von Merkmalsko-Adaptation und die Schaffung eines Ensemble-Effekts.
- **Stärken der L2-Regularisierung:** L2-Regularisierung ist einfacher zu implementieren und recheneffizienter. Obwohl sie allein für sehr tiefe Netze weniger effektiv ist, bietet sie eine stabile und interpretierbare Strafe, indem sie die Gewichtsgrößen begrenzt.
- **Hybridansatz:** Die Kombination beider Methoden führt oft zu einer überlegenen Leistung. L2-Regularisierung begrenzt die Gewichtsgrößen, während Dropout die Merkmalsko-Adaptation verhindert, wodurch ein robustes und generalisiertes Modell entsteht. [18]

5. Praktische Überlegungen Bei der Implementierung von Dropout und L2-Regularisierung sollten mehrere praktische Aspekte berücksichtigt werden:

- **Schichtspezifische Anwendung:** Dropout ist am effektivsten in vollständig verbundenen Schichten, während Faltungsschichten aufgrund ihrer inhärenten räumlichen Lokalität und Parameterteilung weniger profitieren.
- **Hyperparameter-Abstimmung:** Die Beibehaltungswahrscheinlichkeit p für Dropout und die Regularisierungsstärke λ für die L2-Regularisierung sind entscheidende Hyperparameter, die für die besten Ergebnisse durch Techniken wie Kreuzvalidierung sorgfältig optimiert werden müssen.
- **Berechnungsaufwand:** Dropout erhöht zwar die Trainingszeit aufgrund der stochastischen Natur seiner Funktionsweise, hat aber keinen signifikanten Einfluss auf die Inferenzkosten, sobald das Modell trainiert ist. [77]

Fazit

Zusammenfassend lässt sich sagen, dass sowohl Dropout als auch L2-Regularisierung wesentliche Techniken zur Verbesserung der Generalisierungsfähigkeit von tiefen neuronalen Netzwerken sind, insbesondere wenn das Problem der Überanpassung besteht. Dropout führt Stochastizität in den Trainingsprozess ein und zwingt das Modell, robustere und vielfältigere Merkmale zu lernen, während die L2-Regularisierung eine Strafe für große Gewichte auferlegt und so stabilere und glattere Modelle sicherstellt. Obwohl jede Technik ihre eigenen Stärken hat, bietet ihre kombinierte Anwendung eine leistungsstarke Strategie zur Verbesserung der Modellleistung. Wie bei jeder Regularisierungstechnik ist eine sorgfältige Abstimmung der Hyperparameter entscheidend, um optimale Ergebnisse zu erzielen. Das Verständnis der Feinheiten ihrer Anwendung—zum Beispiel der Eignung von Dropout für vollständig verbundene Schichten im Vergleich zu seiner begrenzten Wirksamkeit in Faltungsschichten—kann Praktikern dabei helfen, bessere Modellentscheidungen zu treffen. In der Zukunft wird es wichtig sein, hybride Ansätze zu erforschen, diese Methoden an spezifische Datensätze anzupassen und ihre Anwendung in verschiedenen Netzwerkarchitekturen zu testen, um das Gleichgewicht zwischen Modellkomplexität und Leistung weiter zu verfeinern.